

Prerequisite

The following list the basic prerequisites for installation of a standalone installation of Backstage. It is mandatory to have basic experience of the terminal commands npm, yarn.

- Access to unix based OS, such as Linux, Windows subsystem for Linux (WSL), MacOS.
- Curl or Wget installation
- Node.js LTS release (for our case, Node.js v18.19.1)
- Docker, yarn, Git installation
- GNU-like build environment available at command line

For a start, pull a stable node.js image:

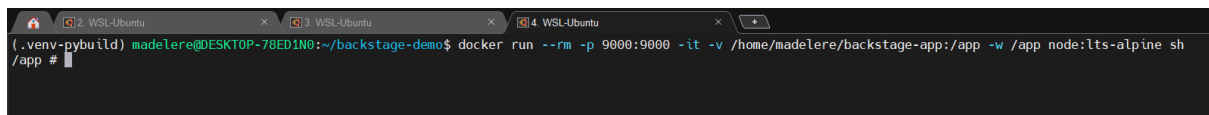
```
docker pull node:lts-alpine (Ensure the choice of node image has npx)
```

```
mkdir -p backstage-app (you can use suitable name of your choice)
```

```
cd backstage-app
```

Now, from your terminal run:

```
docker run --rm -p 9000:9000 -it -v ${PWD}/backstage-app:/app -w /app node:lts-alpine sh
```

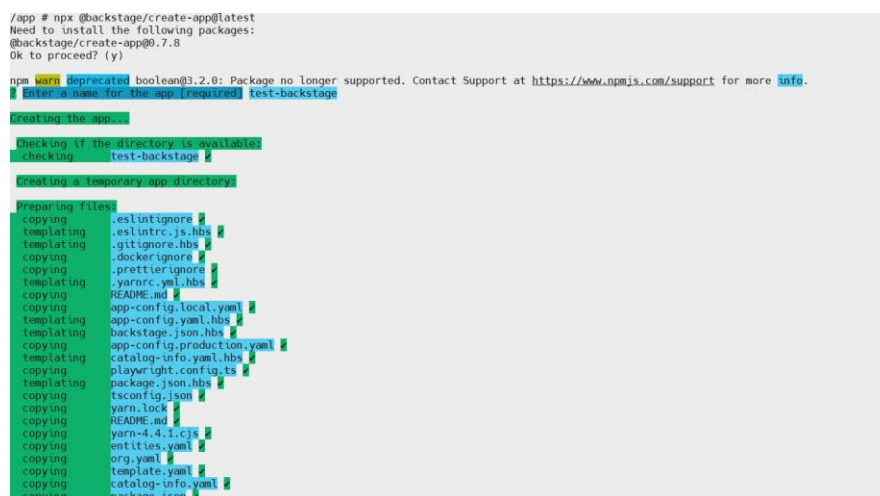


```
WSL-Ubuntu x WSL-Ubuntu x WSL-Ubuntu x
(.venv-pybuild) madelere@DESKTOP-78ED1N0:~/backstage-demo$ docker run --rm -p 9000:9000 -it -v /home/madelere/backstage-app:/app -w /app node:lts-alpine sh
/app #
```

On docker container, run the followings:

```
npx @backstage/create-app@latest
```

A snippet of the output is below:



```
/app # npx @backstage/create-app@latest
Need to install the following packages:
@backstage/create-app@0.7.8
Ok to proceed? (y)

npm warn deprecated boolean@3.2.0: Package no longer supported. Contact Support at https://www.npmjs.com/support for more info.
? Enter a name for the app [required] test-backstage

creating the app...

checking if the directory is available...
checking test-backstage

creating a temporary app directory...

preparing files...
copying .eslintignore
templating .eslintrc.js.hbs
templating .gitignore.hbs
copying .dockerignore
copying .prettierrc
templating .yarnrc.yml.hbs
copying README.md
copying app-config.local.yaml
templating app-config.yaml.hbs
templating backstage.json.hbs
copying app-config.production.yaml
templating catalog-info.yaml.hbs
templating playwright.config.ts
templating package.json.hbs
copying tsconfig.json
copying yarn.lock
copying README.md
copying yarn-4.4.1.cjs
copying entities.yaml
copying org.yaml
copying template.yaml
copying catalog-info.yaml
copying package.json
```

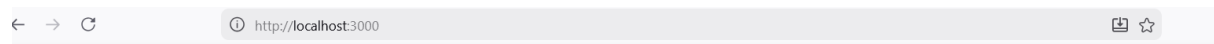
After a successful creation of Backstage, navigate to the project created:

```
cd test-backstage
```

To start the backend, run:

```
yarn start
```

open a browser of choice and try to access e.g. `http://localhost:3000`



The connection was reset

The connection to the server was reset while the page was loading.

- The site could be temporarily unavailable or too busy. Try again in a few moments.
- If you are unable to load any pages, check your computer's network connection.
- If your computer or network is protected by a firewall or proxy, make sure that Firefox is permitted to access the web.

Try Again

In case you have issue like the above, open a new terminal and verify what the problem is:

```
madelere@DESKTOP-78ED1N0:~$ docker ps
```

CONTAINER ID	IMAGE	NAMES	COMMAND	CREATED	STATUS	PORTS
3029a99d0cd5	node:lts-alpine	happy_sutherland	"docker-entrypoint.s..."	4 hours ago	Up 4 hours	0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp

Then:

```
docker exec -it container_id sh
```

```
madelere@DESKTOP-78ED1N0:~$ docker exec -it 3029a99d0cd5 sh
/app #
```

Curl the endpoint to test the connection locally:

```
curl http:localhost:3000
```

Note: for the purpose of this, apk package is use: `apk add curl`

Open `app-config.yaml`, and add

```

/app # curl localhost:3000
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1" />
    <meta name="theme-color" content="#000000" />
    <meta
      name="description"
      content="Backstage is an open source framework for building developer portals"
    />
    <!--
      manifest.json provides metadata used when your web app is installed on a
      user's mobile device or desktop. See https://developers.google.com/web/fundamentals/web-app-manifest/
    -->
    <link
      rel="manifest"
      href="/manifest.json"
      crossorigin="use-credentials"
    />
    <link rel="icon" href="/favicon.ico" />
    <link rel="shortcut icon" href="/favicon.ico" />
    <link
      rel="apple-touch-icon"
      sizes="180x180"
      href="/apple-touch-icon.png"
    />
    <link
      rel="icon"
      type="image/png"
      sizes="32x32"
      href="/favicon-32x32.png"
    />
    <link
      rel="icon"
      type="image/png"
      sizes="16x16"
      href="/favicon-16x16.png"
    />
    <link
      rel="mask-icon"

```

Open app-config, and add

```

GNU nano 8.7
app:
  title: Scaffolded Backstage App
  baseUrl: http://localhost:3000

```

To:

```

GNU nano 8.7
app:
  title: Scaffolded Backstage App
  baseUrl: http://localhost:3000
  listen:
    host: 0.0.0.0

```

Then:

yarn start

Add snippet here

Configuring Github OAuth

From the documentation, available authentication providers range from Auth0 to VMware cloud. The snippet below highlights the list:

- [Auth0](#)
- [Atlassian](#)
- [Azure](#)
- [Azure Easy Auth](#)
- [Bitbucket](#)
- [Bitbucket Server](#)
- [Cloudflare Access](#)
- [GitHub](#)
- [GitLab](#)
- [Google](#)
- [Google IAP](#)
- [Okta](#)
- [OAuth 2 Custom Proxy](#)
- [OneLogin](#)
- [OpenShift](#)
- [VMware Cloud](#)

In our case, GitHub was selected. For follow up, go to [here](#) for the right configuration.

Login to your GitHub, and find your way to *Settings -> Developer settings -> OAuth Apps*

Application name *

backstage

Something users will recognize and trust.

Homepage URL *

http://localhost:3000

The full URL to your application homepage.

Application description

Application description is optional

This is displayed to all users of your application.

Authorization callback URL *

http://localhost:7007/api/auth/github/handler/frame

Your application's callback URL. Read our [OAuth documentation](#) for more information.

Generate a new client secret and copy and save the keys and **client ID** in a secure vault.

The generated provider configuration can be added to **app-config.local.yaml** (development environment) as referenced in GitHub as shown in the snippet below:

```
# Backstage override configuration for your local development environment
app:
  listen:
    host: 0.0.0.0

auth:
  environment: development
  providers:
    github:
      development:
        clientId: ${AUTH_GITHUB_CLIENT_ID}
        clientSecret: ${AUTH_GITHUB_CLIENT_SECRET}
        ## uncomment if using GitHub Enterprise
        # enterpriseInstanceUrl: ${AUTH_GITHUB_ENTERPRISE_INSTANCE_URL}
        ## uncomment to set lifespan of user session
        # sessionDuration: { hours: 24 } # supports `ms` library format (e.g.
        signIn:
          resolvers:
            # See https://backstage.io/docs/auth/github/provider#resolvers f
            - resolver: usernameMatchingUserName
```

```
docker run --rm -e AUTH_GITHUB_CLIENT_ID=xxxxxxxxxxxx -e
AUTH_GITHUB_CLIENT_SECRET=yyyyyyyyyyyyyy -p 3000:3000 -it -p 7007:7007 -v
/home/madelere/backstage-app:/app -w /app node:lts-alpine sh
```

To add Backend installation, go to the root from Backstage then run:

```
yarn --cwd packages/backend add @backstage/plugin-auth-backend-module-github-provider
```

```
/app # cd backstage/  
/app/backstage # yarn --cwd packages/backend add @backstage/plugin-auth-backend-module-github-provider
```

Then add the lines below to `packages/backend/src/index.ts`:

```
backend.add(import('@backstage/plugin-auth-backend'));  
  
backend.add(import('@backstage/plugin-auth-backend-module-github-provider'));
```

```
// auth plugin  
backend.add( import( '@backstage/plugin-auth-backend' ) );  
backend.add( import( '@backstage/plugin-auth-backend-module-github-provider' ) );  
// See https://backstage.io/docs/backend-system/building-backends/migrating#the-auth-plugin  
backend.add( import( '@backstage/plugin-auth-backend-module-guest-provider' ) );  
// See https://backstage.io/docs/auth/guest/provider
```

For full guidelines, goto: <https://backstage.io/docs/auth/github/provider>

To do same for frontend, follow the guideline on here:

<https://backstage.io/docs/auth/#sign-in-configuration>

To add activate resolver, we are going to create a whitelist of users that can access into our backstage. Start with a simple search “backstage GitHub” and navigate to here:

<https://github.com/backstage/backstage/tree/master/packages>

Look for `catalogs-model` -> examples – acmes

master backstage / packages / catalog-model / examples / acme /	
aramissennyeydd code review updates	
Name	Last commit message
..	
backstage-group.yaml	chore: fixing formatting
boxoffice-group.yaml	catalog-model: use relative location targets in examples
infrastructure-group.yaml	catalog-model: use relative location targets in examples
org.yaml	Updated all the Dicebear usages to the proper API URL
team-a-group.yaml	code review updates
team-b-group.yaml	Updated all the Dicebear usages to the proper API URL
team-c-group.yaml	Updated all the Dicebear usages to the proper API URL
team-d-group.yaml	Updated all the Dicebear usages to the proper API URL

For instance, open `team-a-group.yaml` and copy the section for `Users`.

Make a new directory, example:

```
mkdir -p catalog/entities
```

```
cd catalog/entities
```

```
vi users.yaml
```

then add the copied section to the file, the `name` section should be the username of your GitHub account, and save the file.

```

GNU nano 8.7 users.yaml
apiVersion: backstage.io/v1alpha1
kind: User
metadata:
  name: mosesadelere
spec:
  profile:
    email: nigel-manning@example.com
    picture: https://api.dicebear.com/7.x/avataaars/svg?seed=Midnight&backgroundColor=transparent
    memberOf: [team-a]
```

Add the catalog to `app-config.local.yaml`. copy the catalog manifest from `app-config.yaml` and add it in `app-config.local.yaml` just as shown in the snippet below, and update the target with absolute path to the `catalogue/entities/users.yaml`.

```
catalog:
  import:
    entityFilename: catalog-info.yaml
    pullRequestBranchName: backstage-integration
  rules:
    - allow: [Component, System, API, Resource, Location]
  locations:
    # Local example data, file locations are relative to the backend process, typically `packages/backend`
    - type: file
      target: ../../examples/entities.yaml
```

```
GNU nano 8.7 app-config.local.yaml
# Backstage override configuration for your local development environment
app:
  listen:
    host: 0.0.0.0

auth:
  environment: development
  providers:
    github:
      development:
        clientId: ${AUTH_GITHUB_CLIENT_ID}
        clientSecret: ${AUTH_GITHUB_CLIENT_SECRET}
        ## uncomment if using GitHub Enterprise
        ## enterpriseInstanceUrl: ${AUTH_GITHUB_ENTERPRISE_INSTANCE_URL}
        ## uncomment to set lifespan of user session
        ## sessionDuration: { hours: 24 } # supports `ms` library format (e.g. '24h', '2 days'), ISO duration, "human duration" as used in code
      signIn:
        resolvers:
          # See https://backstage.io/docs/auth/github/provider#resolvers for more resolvers
          - resolver: usernameMatchingUserEntityName

catalog:
  rules:
    - allow: [User, Component, System, API, Resource, Location]
  locations:
    # Local example data, file locations are relative to the backend process, typically `packages/backend`
    - type: file
      target: /app/backstage/catalog/entities/users.yaml
```

To test what has been done, start the server:

Yarn start

Then open a browser and enter <http://localhost:3000>. At the prompt of the page shown below, login with your GitHub credentials.

